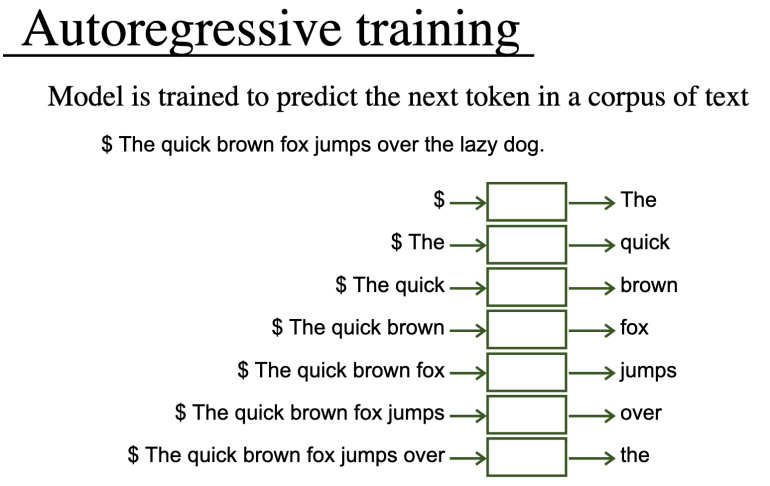
LECTURE 13

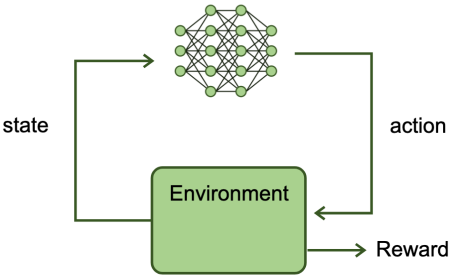
Program Synthesis with Reinforcement Learning

Up to this point, we have talked about how language models are trained using supervised learning with an autoregressive objective.

There is however another way to train a model to produce programs, and that is by using *reinforcement learning* (RL). Unlike the supervised learning approach, RL does not require a corpus of programs to train on. Instead, the idea is that the model has access to an environment that produces rewards or penalties for every action that the model takes. This allows for a form of self-training, where the model discovers the best (i.e., most rewarding) actions to take in different situations.



Reinforcement learning



Network is scored by how it behaves in the context of an environment
No supervision about individual inputs/outputs to the network
No need to prescribe specific outputs





13.1 A reinforement learning primer

In reinforcement learning, the goal is to learn how to control an agent interacting with a possibly non-deterministic environment. The environment is defined by a state space $\mathcal{S}$ and a transition function $T : S \times \mathcal{A} \times S \to \mathbb{R}$. Given a source state s an action a and a target state s', the function $T(s, a, s')$ is equal to the probability that if the agent in a state s and it performs an action a, that it will transition to state s'. In order for T to be well formed, it must be the case that for all initial states and all actions, the sum of T over all target states should add up to 1.

$$\forall s. \forall a. \left(\sum_{s'} T(s, a, s') \right) = 1$$

In most practical applications of RL, the environment is represented with a simulator— a complex piece of code that is not differentiable or analyzable and that internally keeps track of the state of the system. For example, if we want to train an agent to play pong, the state would be the position and velocity of the ball and the position of the paddles. The actions would correspond to the commands to move the paddle up or down, and the transition function would be the simulator that determines the next state from the current state and the action. The uncertainty in pong comes from the adversary, which may behave non-deterministically.

The goal in RL is to learn a policy $\pi : \mathcal{S} \to \mathcal{A}$ that for each state determines an action for the agent and which maximizes a reward $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ that

assigns a score to each state action pair. The policy can also be probabilistic, in which case $P_\pi : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ tells us the probability of taking an action a in state s. In most interesting applications of RL, the reward is sparse; only a handful of state/actions have non-zero rewards. For example, in pong, you may get a reward every time your paddle hits the ball. In the case of some games, the reward may be zero until you reach the end of the game—only at that point you get a positive or negative reward depending on whether you won or lost.

A trajectory or rollout $\tau = (s_0, a_0, s_1, a_1, \ldots, s_n)$, is a sequence of states and actions starting from an initial state s_0. The reward of a rollout (Cumulative discounted reward) is $R(\tau) = \sum_i^{n-1} \gamma^i R(s_i, a_i)$. The discount factor $\gamma < 1$ is used to give more importance to immediate rewards than to rewards far into the future. This is especially important in infinite horizon games, where the agent is expected to play forever. The probability of a rollout given a policy is $P_\pi(\tau) = P(s_0) \cdot \prod_{i=0} P_\pi(a_i|s_i) \cdot T(s_i, a_i, s_{i+1})$

In learning a policy, it is often useful to compute a *value function*. A value function $V_\pi(s)$ computes the expected reward if you follow a policy π starting from state s. This means the value function will satisfy the following recurrence relation:

$$V_\pi(s) = R(s, \pi(s)) + \gamma * \left(\sum_{s'} T(s, \pi(s), s') * V_\pi(s') \right)$$

So the reward at state s is the reward for the action we take at state s plus the discounted reward from the states we transition to from s.

Value function

$$V^\pi(s) = E_{\tau \sim \pi, s}[\ R(\tau)\ ]$$

Goal: Maximize $E_{s \sim P}[V^\pi(s)]$

Bellman equation with deterministic policy:

$$V^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s'} T(s, \pi(s), s') * V^\pi(s')$$

Bellman equation (general case):

$$V^\pi(s) = \sum_a P_\pi(a|\ s) * (\ R(s, a) + \gamma \sum_{s'} T(s, a, s') * V^\pi(s'))$$

A simple RL algorithm

Value iteration

$$V^*(s) = \max_a R(s,a) + \gamma \sum_{s'} T(s,a,s') * V^*(s')$$

Initialize $V^*(s)$=0 and then iteratively update it using the equation above

Then compute the policy as

$$\pi(s) = \operatorname{argmax}_a R(s,a) + \gamma \sum_{s'} T(s,a,s') * V^*(s')$$

An alternative simple RL algorithm

Policy iteration

Initialize πrandomly

Update $V^{\pi}(s)$for the given π

$$V^{\pi}(s) = R(s,\pi(s)) + \gamma \sum_{s'} T(s,\pi(s),s') * V^{\pi}(s')$$

Then compute the policy as

$$\pi(s) = \operatorname{argmax}_a R(s,a) + \gamma \sum_{s'} T(s,a,s') * V^{\pi}(s')$$

Another useful function when learning a policy is the *action value function*, also often refered to as the Q function. $Q_\pi : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$. The function $Q_\pi(s,a)$ computes the expected reward if starting from state s, you first take action a and subsequently just follow the policy π.

$$Q_\pi(s,a) = R(s,a) + \gamma * \left(\sum_{s'} T(s,a,s') * V_\pi(s') \right)$$

When T is deterministic, we can use the shorthand $T(s,a)$ to refer to the state from which T transitions from s on action a with probability one. In that case, V and Q simplify to

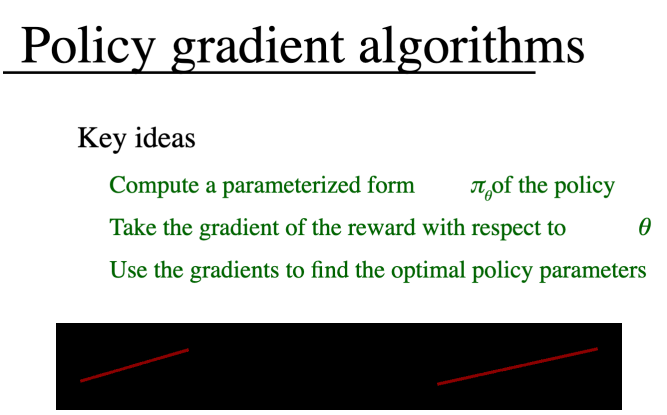
$$V_\pi(s) = R(s,\pi(s)) + \gamma * V_\pi(T(s,\pi(s)))Q_\pi(s,a) = R(s,a) + \gamma * V_\pi(T(s,a))$$

Two simple algorithms for reinforcement learning are Value Iteration and Policy Iteration. Both of these algorithms iteratively improve the policy and the value function until they converge to an optimal policy π^* and an optimal

value function V^*. However, both algorithms are only practical when the state space is small enough to explicitly represent the value function and the policy as tables. In most practical applications of RL, the state space is too large for an explicit representation. Today, most practical RL algorithms use neural networks to represent the policy and value functions, so the key is to define an optimization problem that can be solved without having to propagate gradients through the environment simulator.

13.2 Policy Gradient Methods

An important class of algorithms for reinforcement learning are policy gradient methods. In this section, we will describe the REINFORCE algorithm [1], which is one of the simplest policy gradient methods, and was used in several early applications of reinforcement learning to program synthesis.



In this algorithm, the goal is to find an optimal parameter for the policy that maximizes the expected reward for the policy. In other words

$$\theta^* = argmax_\theta \, \mathbb{E}_{\tau \sim P_{\pi_\theta}(\tau)}[R(\tau)]$$

In order to compute θ^*, we can use gradient ascent, but this requires us to compute an expression for the gradient of the expected reward with respect to θ, and we want to do this without having to differentiate through the environment simulator.

The figure on the right shows the full derivation for the gradient of the expected reward with respect to θ. The final result is that the gradient can be expressed as an expectation over trajectories for the current policy π_θ.

$$\nabla_\theta \mathbb{E}_{\tau \sim P_{\pi_\theta}(\tau)}[R(\tau)] = \mathbb{E}_{\tau \sim P_{\pi_\theta}(\tau)} \left[R(\tau) \cdot \sum_{t=0}^{n-1} \nabla_\theta \log P_\pi(a_t | s_t) \right]$$

This means that we can estimate the gradient by sampling trajectories from the current policy, computing the reward for each trajectory, and then computing the sum of the gradients of the log probabilities of the actions taken in the trajectory weighted by the reward.

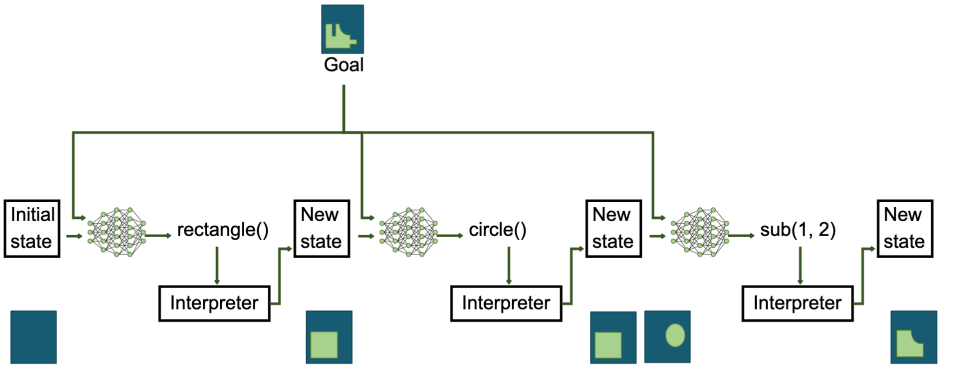
13.3 Reinforcement learning for Program Synthesis

The REINFORCE algorithm has been applied with some success to program synthesis problems, but even after chosing a synthesis algorithm, there are other important design decisions including the choice of what is the state space and what are the actions that the agent can take.

The simplest way to formulate program synthesis as a reinforcement learning problem is to define the state space S as the set of partial programs and the actions correspond to growing the program. For example, in the context of top-down search, an action correspond to selecting a hole in the program and expanding it using one of the available rules. For languages where programs correspond to linear sequences of instructions, an action can simply correspond to appending an instruction at the end of the partially constructed program. In this context, the reward is sparse, all states corresponding to partial programs have a reward of zero, and states corresponding to completed programs have positive or negative rewards depending on whether the program is correct or incorrect.

The first paper to take this view was written by Bunel, Hausknecht, Devlin, Singh and Kohli [2]. Their result was improved by Chen, Liu and Song [3] who observed that by incorporating the state of the execution as part of the state, you could leverage the program interpreter to give the policy a better sense of what the program so far can actually compute.

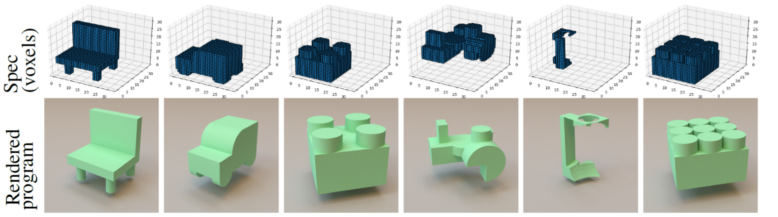
Synthesis with REPL



K. Ellis, M. Nye, Y. Pu, F. Sosa, J. Tenenbaum, A. Solar-Lezama, *Write, Execute, Assess: Program Synthesis with a REPL*, NeurIPS 2019

2D and 3D reconstruction

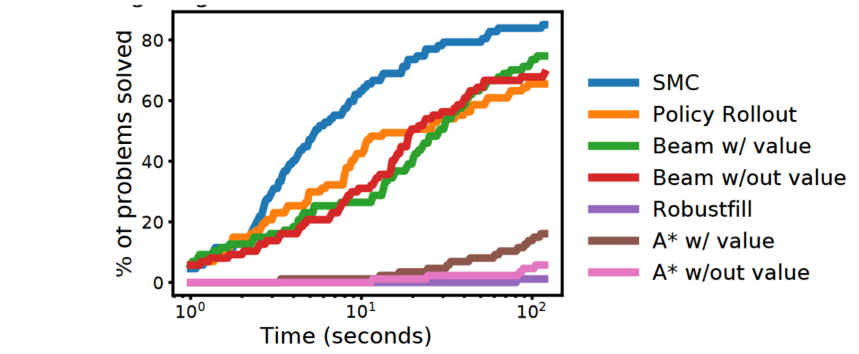




REPL on string transformations

Spec:	
6/12/2003	→ date: 12 mo: 6 year: 2003
3/16/1997	→ date: 16 mo: 3 year: 1997
Held out test instance:	
12/8/2019	→ date: 8 mo: 12 year: 2019
Results:	
SMC (Ours)	→ **date: 8 mo: 12 year: 2019**
Rollout	→ date: 8 mo: 1282019
Beam w/value	→ date: 8 mo: 12 year:2019
Beam	→ date: 8 mo: 12 year:
RobustFill	→ date:12/8/2019

Spec:	
Dr Mary Jane Lennon	→ Lennon, Mary Jane (Dr)
Mrs Isaac McCormick	→ McCormick, Isaac (Mrs)
Held out test instance:	
Dr James Watson	→ Watson, James (Dr)
Results:	
	→ **Watson, James (Dr)**
	→ Watson, James
	→ Watson, JamesDr
	→ Watson, James (
	→ Dr James Watson



In 2019, Ellis, Nye, Pu, Sosa, Tenenbaum and Solar-Lezama [4] took this observation one step further by making the state be just the program state (the values of the variables) and not include the program text at all, as illustrated in the figure. The intuition for this is the same as the intuition behind observational equivalence from Lecture 3—programs that produce the same outputs on the given inputs do not have to be distinguished, and collapsing them into an equivalence class helps with symmetries. Thus, in this formulation, the state s is the state computed by the program so far, and an action is an instruction that transforms this state to a new state. The algorithm uses REINFORCE to learn a policy that generates programs instruction by instruction, and also trains a neural network to estimate the value function. Learning a value function means that in addition to the sampling techniques that we explored in Lecture 11, we can also use the value function to guide how the algorithm samples programs.

In addition to the choice of state and action space, there are other important considerations in applying reinforcement learning to program synthesis. One important observation is that the sparse reward signal, combined with the large state and action spaces, makes the early stages of learning very inefficient, since the model will have to sample extensively before it finds any samples with positive reward. One way many papers have addressed this issue is by pre-training the model using supervised learning on a corpus of programs, and relying on reinforcement learning only to fine tune the model, but having a training set with a variety of programs at different levels of difficulty can also help with this problem.

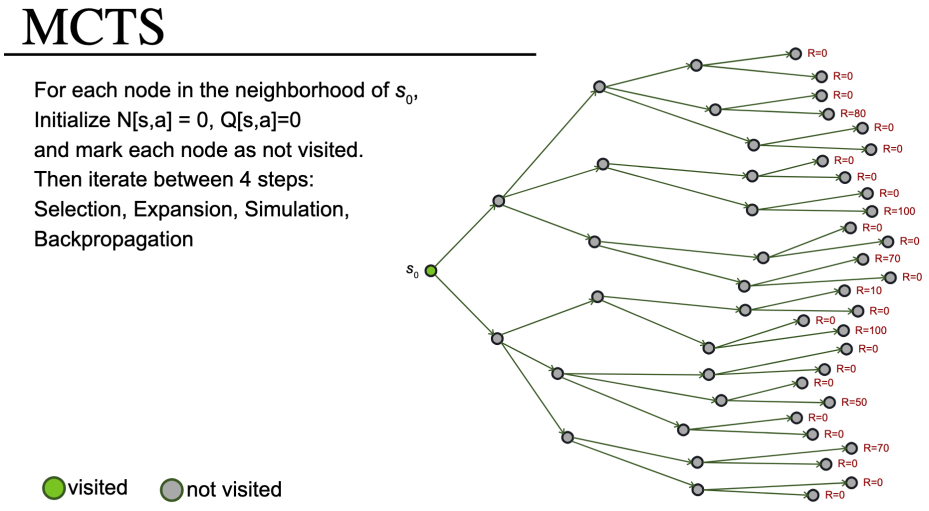
Another consideration when using LLMs with RL for program synthesis is that the tradeoffs can sometimes be different from those cases where the LLM is used by itself. For example, in pure LLM-based synthesis, it is often desirable to have a very large model since this tends to improve the quality of the generated programs. However, in RL-based synthesis, having a very large model

can make learning more difficult, since it makes sampling substantially more expensive.

13.4 Basic MonteCarlo Tree Search (MCTS) and AlphaZero

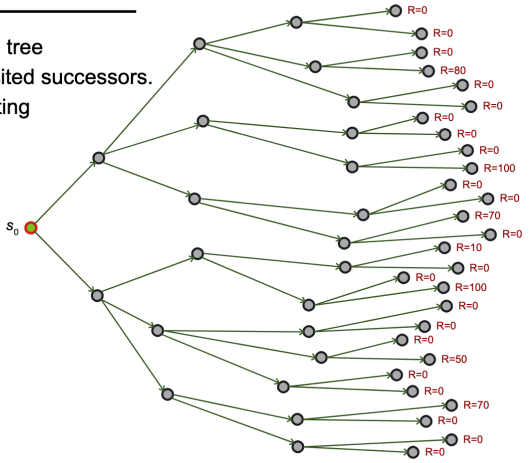
Since those early applications of RL to program synthesis mentioned earlier, reinforcement learning has become much more prominent. DeepMind, in particular, is well known for its work on reinforcement learning for games, and they have adapted some of the same algorithms to program synthesis. One of their workhorse algorithms is AlphaZero [5], which is based on an algorithm known as MonteCarlo Tree Search (MCTS) [6].

The goal of AlphaZero is to find a probabilistic policy $P : \mathcal{S} \rightarrow \mathcal{A} \rightarrow \mathbb{R}$ that for every state s and every action a, produces a value between zero and one representing the probability of taking action a at state s. So for example, we can compute a policy π that always takes the highest probability policy as $\pi(s) = argmax_a P_\theta(s, a)$, although as we shall see, the computation of the actual actions to take at a given state will be a little more involved. Additionally, the algorithm will compute a value function $V_\pi(s)$ that estimates the reward that the policy will achieve starting at state s.



MCTS

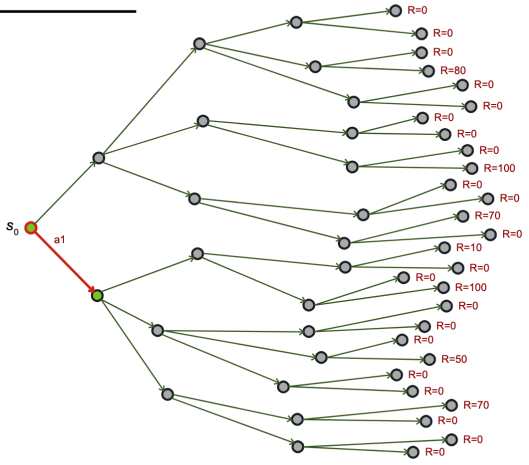
Selection: Use N and Q to traverse the tree until you reach the first node with unvisited successors. The firs time around is not very interesting because that's just s_0.



visited not visited

MCTS

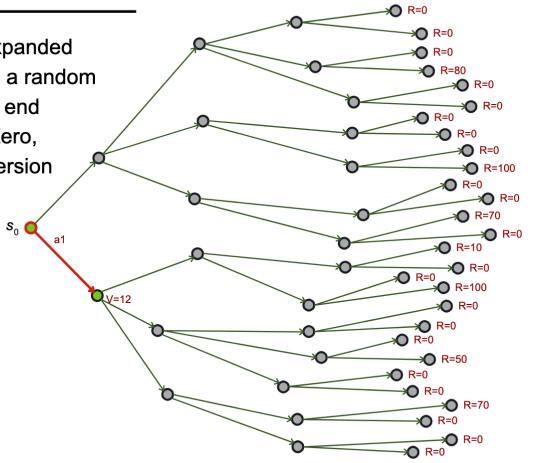
Expansion: Randomly select a child of the current node.



visited not visited

MCTS

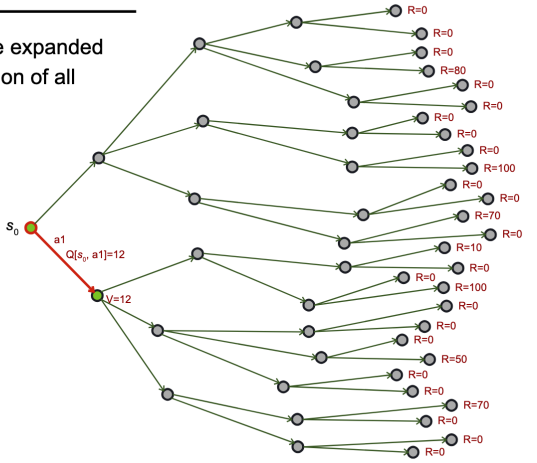
Simulation: Estimate a value for the expanded node. This can be done by just running a random path from that node until you reach the end and you collect a reward, but in AlphaZero, this estimate comes from the current version of the value function.



visited not visited

MCTS

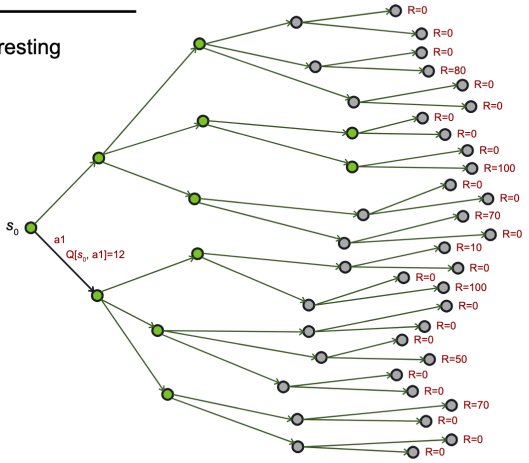
Backpropagation: Use the value of the expanded node *e*and use it to update the Q function of all the nodes in the path from $s_0$to *e*.



visited not visited

MCTS

The Selection step becomes more interesting when more nodes have been visited.



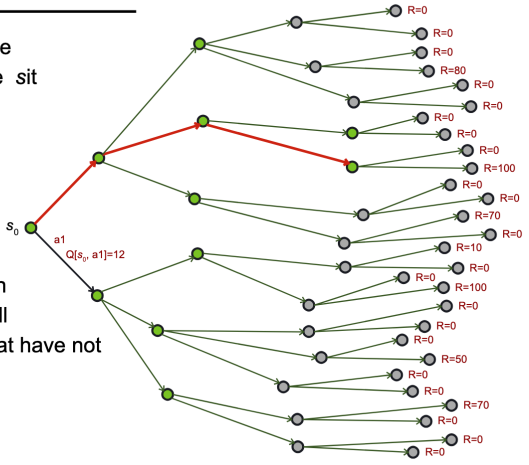
visited not visited

MCTS

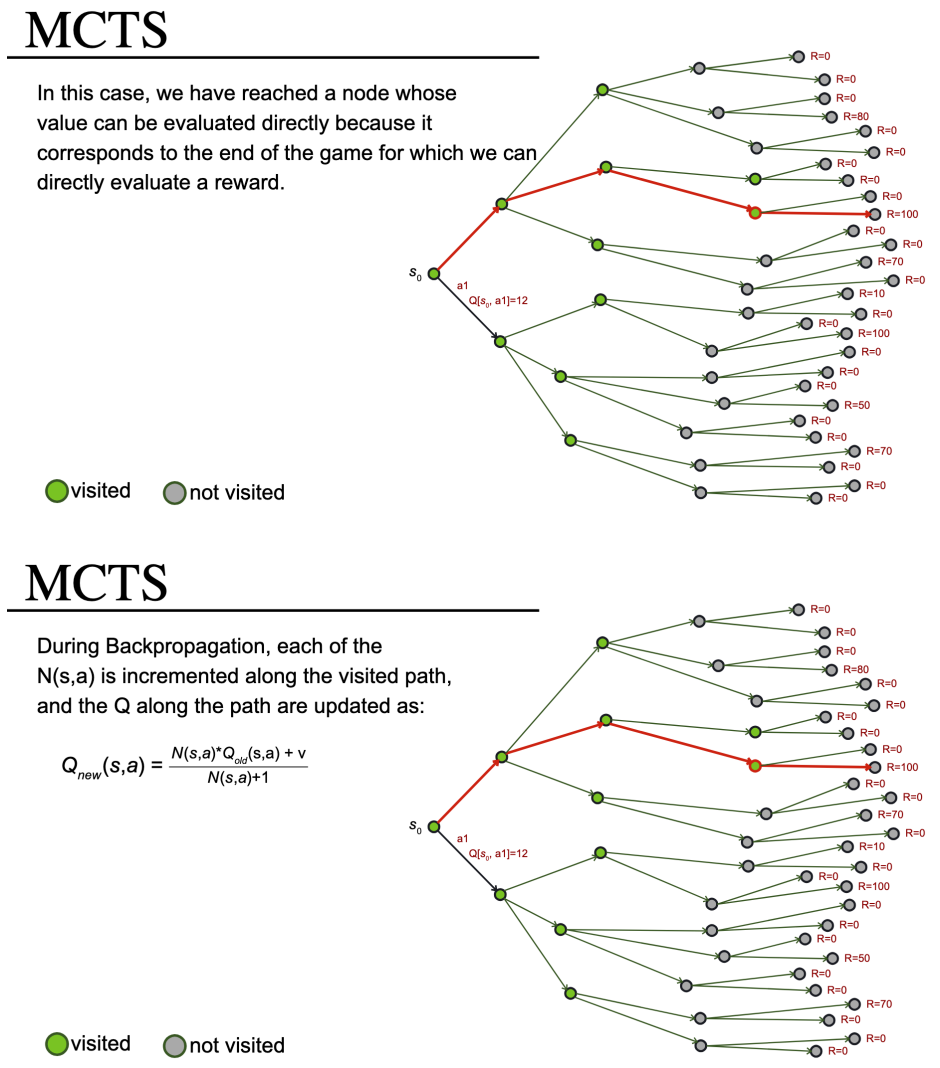
In that case, selection traverses the tree by starting at $s_0$and then for every state $s$it picks an action defined by:

$$argmax_a\ Q(s,a) + c_p \cdot P(s,a)\ \frac{\sqrt{N(s)}}{1+N(s,a)}.$$

The $c_p$parameter balances exploration and exploitation if $c_p$is zero, the network will constantly chose the action that maximizes Q, but if $c_p$is large, it will be biased towards exploring actions that have not been tried already.



visited not visited



The algorithm is based on deep learning, so the probabilistic policy and value functions will be represented by parametric functions P_θ and V_θ. In practice, P and V are represented by a single neural network and θ is a very large collection of parameters that controls both functions, but that is a low-level implementation detail. The goal of the learning algorithm is to discover a good parameter θ such that the policy achieves high rewards and the Value function V_θ accurately predicts the reward of following the policy for the reminder of the game starting at state s.

The key building block in the algorithm is MCTS. MCTS performs some local exploration in the neighborhood of a state to discover improved estimates of P and V which can then be used to adjust θ.

The basics of MCTS are illustrated in the figure. In the algorithm, you start at a state s_0 and iteratively search the space guided by an approximation of the Q function built on the fly. The algorithm maintains a map representing the Q function $Q(s, a)$ and a counter $N(s, a)$ that tracks how many times action a has been taken from state s. We use the shorthand $N(s)$ to refer to the total number of times state s has been visited $N(s) = \sum_a N(s, a)$.

The algorithm is traditionally explained in terms of four phases:

Selection. Traverses a path through the visited nodes in search of a node at the boundary between visited and not-visited nodes. In searching for this path, the algorithm uses the following recursive formula:

$$search(s) = \begin{cases} search(T(s, action)) \text{ where } action = argmax_a \left(Q(s,a) + C * P(s,a) * \frac{\sqrt{N(s)}}{1+N(s,a)} \right) \\ \\ s \text{ if s has unvisited children} \end{cases}$$

In other words, in taking an action for each state where all its children have been visited, it attempts to maximize a quantity that considers both the Q value of the proposed action, as well a measure of how desireable the current policy regards action a adjusted by a measure of how many times the current action has been explored relative to other actions available at this level. The constant C is used to give more or less weight to the second component relative to the first.

Expansion. Once selection reaches a boundary between visited and non-visited nodes, a new node is selected that has not been visited before and its Q and N measures are initialized.

Simulation. In this step, the system computes an estimate of the value of the currently expanded node. This can be done in many different ways, but in the case of AlphaGo/AlphaZero, the value is red directly from the last version of the value function, unless the end of the game has been reached, in which case the game is just scored explicitly.

Backpropagation Once a value has been computed for the node, this value can be used to compute an estimate of Q for each node in the path from the origin to the expanded node. For example, if values are not discounted over time (i.e. $\gamma = 1$), then this just means updating $Q(s, a)$ by adjusting the old value of Q to $(Q(s, a) * N(s, a) + V)/(N(s, a) + 1)$.

Those steps are repeated over and over again a few hundred times. After this is done enough times, the function Q can be used to provide an improved estimate of both the value function and the policy for the nodes visited during search. The parameters for P and V can then be adjusted to get them closer to these improved estimates. By iteratively repeating this process of running MCTS and then adjusting the parameters to more closely match the P and V

suggested by the Q function, the algorithm converges to a good policy and value function.

References

[1] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3-4):229–256, 1992. doi: 10.1007/BF00992696. URL https://doi.org/10.1007/BF00992696.

[2] Rudy Bunel, Matthew J. Hausknecht, Jacob Devlin, Rishabh Singh, and Pushmeet Kohli. Leveraging grammar and reinforcement learning for neural program synthesis. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net, 2018. URL https://openreview.net/forum?id=H1Xw62kRZ.

[3] Xinyun Chen, Chang Liu, and Dawn Song. Execution-guided neural program synthesis. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019. URL https://openreview.net/forum?id=H1gfOiAqYm.

[4] Kevin Ellis, Maxwell I. Nye, Yewen Pu, Felix Sosa, Josh Tenenbaum, and Armando Solar-Lezama. Write, execute, assess: Program synthesis with a REPL. In Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pages 9165–9174, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/50d2d2262762648589b1943078712aa6-Abstract.html.

[5] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018. doi: 10.1126/science.aar6404. URL https://www.science.org/doi/abs/10.1126/science.aar6404.

[6] Rémi Coulom. Efficient selectivity and backup operators in monte-carlo tree search. In H. Jaap van den Herik, Paolo Ciancarini, and H. H. L. M. (Jeroen) Donkers, editors, *Computers and Games*, pages 72–83, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg. ISBN 978-3-540-75538-8.